

fMRI-DP — Final Diagnosis and Fix Plan

THE ISSUE

Functional connectivity never reaches the graph convolution. Anywhere. Not during training, and not during evaluation.

Where it breaks — training

`GInfoMinMax.forward` signature:

```
forward(batch, x, edge_index, beta, edge_attr, gcn_edge_weight, pcc_weight,
dyn_weight)

                                ^^^^^^^^^^^^^^^^^^  ^^^^^^^^^^^^^
                                goes to the GCN    goes to

ToyNet only
```

The four call sites:

Line	View	<code>gcn_edge_weight</code>	Result
191	original	<code>None</code>	every edge = 1.0
209	augmented	<code>batch_aug_edge_weight</code>	learned gate only
251	original	<code>None</code>	every edge = 1.0
268	augmented	<code>batch_aug_edge_weight</code>	learned gate only

`batch.edge_weight` — the 90x90 FC matrix — is only ever passed in the `pcc_weight` slot, which feeds `ToyNet.get_mu_std_logits` and nothing else. Grep confirms: no call site anywhere passes it as `gcn_edge_weight`.

Where it breaks — evaluation (worse)

`TA_encoder.py:338`, inside `get_embeddings`:

```
python

edge_weight = None
```

Hardcoded. Every embedding used for the SVM is computed on a completely unweighted graph. So even if training were fixed, the reported accuracy would not move — the classifier never sees a connectivity-informed embedding.

Why this produces near-chance accuracy

`edge_index` is all 8100 pairs (complete graph, built by meshgrid). With `edge_weight = None`, GCNConv assigns 1.0 to every edge.

Therefore **every subject's graph is byte-identical**: same 90 nodes, same complete topology, same uniform weights. Subject 1's graph equals subject 956's graph.

The only per-subject information entering the encoder is $\mathbf{x}$ — $90 \times 3 = 270$ ALFF numbers. The GCN then averages those over a complete graph, which is close to computing a global mean and discarding spatial structure.

You have an ALFF-only classifier wearing a GNN costume. Chance-level performance is the correct outcome for that model.

Why the code comment is wrong

`ginfo_minmax.py:30` and `TA_encoder.py:335` both justify this as "Eq. (17): $\mathbf{X_Topo} = \text{GCN}(\mathbf{X}, \mathbf{A})$, the unweighted adjacency -- not $\mathbf{W}$."

The concern behind it is real: raw signed PCC breaks GCNConv's $\text{deg}^{-1/2}$ normalization, because negative weights can make the degree zero or negative.

But the conclusion does not follow. In the paper, **A is the learned adjacency** — Eq. 12 soft-sparsifies $\mathbf{W}$ into $\mathbf{A}$, and Section 3.2 says explicitly: *"instead of discarding certain edges, we weaken their connections."* Weakening requires something to weaken. An all-ones matrix is not $\mathbf{A}$; it is $\mathbf{A}$ before any learning has happened, frozen forever.

The correct response to the sign problem is `abs()`, not `None`.

Cross-check

A-GCL (Zhang et al. 2023) uses the same 3-band ALFF node features and the same PCC edge weights on the same ABIDE-I data, and reaches 80.65%. Their GCN aggregates over real FC weights. That is the difference.

Your preprocessing is not the problem — the atlas averaging is validated at $r = 1.0000$.

THE FIX

1. `GraSTIACL.py` — four arguments

Add once, immediately after `batch = batch.to(device)`:

```
python

gcn_w = batch.edge_weight.abs().clamp(1e-6, 1.0)
```

Then change the 6th positional argument:

```
python
```

```
# lines 191 and 251 -- original view
x, _, edge_logits = model(batch.batch, batch.x, batch.edge_index, beta, None,
                           gcw_w,
                           batch.edge_weight, batch.dyn_weight)

# lines 209 and 268 -- augmented view
x_aug, _, _ = model(batch.batch, batch.x, batch.edge_index, beta, None,
                    gcw_w * batch_aug_edge_weight,
                    batch.edge_weight, batch.dyn_weight)
```

The multiplication is the point: `(gate x FC)` is a weakened connection (Eq. 12's soft dropout). `gate` alone throws the connection away and substitutes a learned number.

2. `TA_encoder.py:338` — the evaluation path

```
python

# was: edge_weight = None
edge_weight = data.edge_weight.abs().clamp(1e-6, 1.0)
```

Do not skip this. Fixing only training leaves your reported accuracy unchanged, and you will wrongly conclude the diagnosis was wrong.

3. Check gamma is no longer degenerate

`TA_encoder.py:294-297`: when `(edge_weight is None)`, gamma is hardcoded to `(1 - eps)`, so `Beta(0.9999, 0.0001)` puts $\lambda \sim 1$ for the original view while the augmented view gets a real gamma — a systematic asymmetry between the two contrastive views.

After steps 1-2 both views have genuine weights, so this resolves itself. Print gamma for both views once to confirm it is sensible and no longer pinned at 1.

VERIFY BEFORE TRUSTING ANYTHING

Gate A — the shuffle test

Randomly permute `(edge_weight)` across subjects, keep labels fixed, rerun.

- **Accuracy collapses toward 50%** -> connectivity is now used. Proceed.
- **Accuracy unchanged** -> it still is not reaching the encoder. Stop.

This single run tells you whether the diagnosis was right. Run it first.

Gate B — views must differ

Print `(x - x_aug).abs().mean()` for one batch. Near zero means InfoNCE has near-identical positives and almost no gradient signal.

Gate C — gate must not saturate

Print `batch_aug_edge_weight.mean()` and `.std()` per epoch. Collapse to all-0 or all-1 means augmentation has died.

THEN RERUN

5-fold stratified, seed 123, current defaults.

Result	Meaning	Next
70-80%	Fixed	Run ablations, LOSO, window sweep
60-70%	Mostly fixed	Tune <code>ce_lambda</code> , <code>kld_lambda</code> , <code>emb_dim</code>
Still ~50%	Something else	Report gates A/B/C output

`kld_lambda=0.003` and `ce_lambda=2.0` are the paper's values for ADNI/SRPBS/BD. With real gradients now flowing through connectivity, these may need adjusting. That is tuning, not a bug.

HONEST CAVEAT

This is a confirmed bug — the grep is decisive, `batch.edge_weight` never enters `gcn_edge_weight`, and `get_embeddings` hardcodes `None`. Fixing it will change your results.

It is not a guarantee of 80%. Two known limitations remain, both already documented and accepted in your `ISSUES.md`:

- The encoder trains on all 956 subjects before the SVM's fold split (no nested CV). Inflates results rather than suppressing them, so it is not the cause of low accuracy — but it is a limitation to state in the paper.
- Loss weights have never been tuned with real gradients flowing through connectivity.

Run Gate A first. It answers the question in one run.

WHAT IS ALREADY CORRECT — DO NOT TOUCH

- ToyNet `FC_mean/FC_var` map `hidden -> latent` directly; the 1x1/90x800 crash is gone
- `get_mu_std_logits` concatenates static (90) + dynamic (270) = 360 per ROI; dynamic weights genuinely receive gradient
- `WindowedData.__cat_dim__` stacks `dyn_weight` to `[B, T, 90, 90]`
- Gate now built from `edge_logits_v1` (the view learner's own output)
- `edge_index` = meshgrid, all 8100 pairs; immune to the exact-zero crash
- Subjects matched by filename, not `os.listdir()` position
- Labels ASD=1, NC=0; metrics describe patients

- Attention block-diagonal per subject; $\text{softmax}(QK^T/\sqrt{d})$ matches Graphormer Eq. 4
- `drop_last=False` with `batch.num_graphs` throughout
- StratifiedKFold seeded, scaler inside GridSearchCV, AUC from `decision_function`, all metrics read from `best_val_epoch`
- Preprocessing validated at $r = 1.0000$